COMPREHENSIVE UVM-BASED VERIFICATION OF ALSU USING DUAL ENVIRONMENTS

PROJECT DONE BY :

YOUSEF SHERIF ALI HASSAN ELGENDY

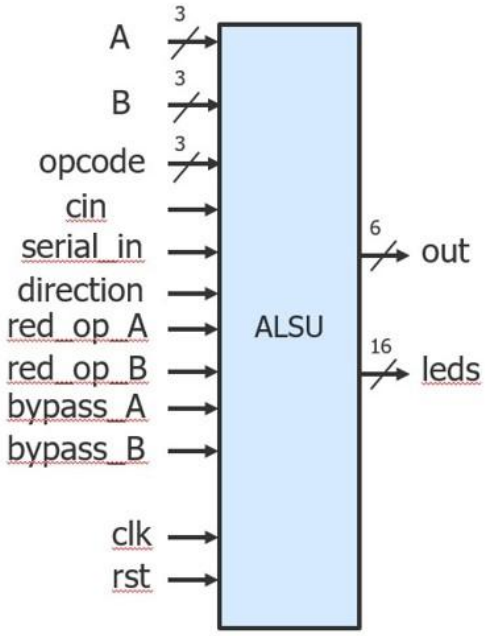
Introduction :

- The **ALSU** design was enhanced to support **Shift** and **Rotate** operations using a separate **Shift Register module**.
- Instead of instantiating the Shift Register directly inside the ALSU, **SystemVerilog interfaces** were used to connect both modules.
- The **interfaces** handle signal communication between the ALSU and the Shift Register, improving **modularity**, **scalability**, and **reusability** of the design.
- The **ALSU** controls when the Shift Register is active, depending on the **opcode** or operation mode.
- This approach allows both modules to operate **independently** while maintaining **synchronization** through shared interface signals (e.g., `clk`, `rst`, `serial_in`, `direction`, etc.).
- Two **UVM environments** were developed:

ALSU Environment – verifies arithmetic and logical functionalities.

Shift Register Environment – verifies shifting and rotation functionalities.

- Both environments are connected to their respective **interfaces** and are verified together under a **single UVM test**.
- This integrated UVM setup enables **parallel verification** of both modules while maintaining **clear separation of responsibilities**.
- The testbench ensures **functional coverage** across all operation types and validates the **correct interface communication** between modules.
- This structure reflects a **multi-environment UVM testbench architecture**, demonstrating a scalable verification approach suitable for **SoC-level integration**.



Verification Plan

Label	Design requirement Description	Stimulus generation	Functional coverage	Functionality check
ALSU_1	When the reset is asserted, the output counter value should be low and leds are zero	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.	-	**IMMEDIATE ASSRTION +** A checker in the testbench to make sure the output is correct
ALSU_2	In case of invalid not occur When the opcode is addition, output should perform addition on A ,B taking cin if Parameter FULL_ADDER is high	Randomization under constraint on A, B to take the values (MAXPOS, ZERO and MAXNEG) more often than the other values	Cover A and B at (MAXPOS , ZERO , MAXNEG) , and bin for add or mult , and Transition bin from opcode 0 > 1 > 2 > 3 > 4 > 5	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_3	In case of invalid not occur When the opcode is mult, output should perform multiplication on A , B if reduction_A and reduction_b are low	Randomization under constraint on A, B to take the values (MAXPOS, ZERO and MAXNEG) more often than the other values	Cover A and B at (MAXPOS , ZERO , MAXNEG) , and bin for add or mult , and Transition bin from opcode 0 > 1 > 2 > 3 > 4 > 5	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_4	In case of invalid not occur When the opcode is OR, output should perform OR Operation on A ,B if reduction_A and reduction_b are low	Randomization without constraints on A, B but reduction ports are disabled most of the time	Cover A and B at (MAXPOS , ZERO , MAXNEG) , and bin for OR or XOR, and Transition bin from opcode 0 > 1 > 2 > 3 > 4 > 5	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_5	In case of invalid not occur When the opcode is OR & any of the inputs reduction_A or reduction_b is high , output should perform Reduction_OR Operation on A ,B based on priority	Randomization under constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be IOW if reduction_A is high.	Cover A at (A_data_walkingones) if reduction_A is high while B is zero,or Cover B at (B_data_walkingones) if reduction_B is high and reduction_A is low, while A is zero, and bin for OR or XOR , and Transition bin from opcode 0 > 1 > 2 > 3 > 4 > 5	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_6	Incase of invalid cases do not occur, when opcode is XOR, then out Should perform the XOR operation on ports A and B if reduction_A and reduction_B inputs are low	Randomization without constraints on A or B. reduction ports are disabled most of the time	Cover A and B at (MAXPOS , ZERO , MAXNEG) , and bin for OR or XOR , and Transition bin from opcode 0 > 1 > 2 > 3 > 4 > 5	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_7	Incase Of invalid cases do not occur, when opcode is XOR & any of the inputs reduction_A or reduction_B is high, then out should perform the reduction XOR operationA on ports A and B based on the priority	Randomization under constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be IOW if reduction_A is high.	Cover A at (A_data_walkingones) if reduction_A is high while B is zero . or Cover B at (B_data_walkingones) if reduction_B is high and reduction_A is low while A is zero, and bin for OR or XOR , and Transition bin from opcode 0 > 1 > 2 > 3 > 4 > 5	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_8	Incase of invalid cases do not occur, when opcode is SHIFT then the output out will shift right or left based on the direction Input	Randomization without constraints	Cover the SHIFT	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_9	Incase of invalid cases do not occur, when opcode is ROTATE then the output out will rotate right or left based on the direction Input	Randomization without constraints	Cover the ROTATE	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_10	When invalid cases exist without bypass, out output should be low and leds should blink	Randomization under constraints where invalid cases do not occur as frequent as valid cases	Cover the Illegal opcode values + Transition bin from opcode 0 > 1 > 2 > 3 > 4 > 5	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_11	If invalid cases do not occur and the bypass inputs are high, then the output out should by bypass port A or B based on the Prioirty if the both bypass ports are high	Randomization under constraint on bypass ports to be disabled most of the time	----	**CONCURREMT ASSRTION +** Output checked against GoldenModel
ALSU_12	In case of valid opcode . out should be evaluated according to the operation	Randomization under valid opcodes while looping on different opcodes each cycle to be disabled most of the time	Cover the opcode values using legal, bins as well as transition to make sure output is evaluated at each opcode change with each transition : Transition bin from opcode 0 > 1 > 2 > 3 > 4 > 5	**CONCURREMT ASSRTION +** Output checked against GoldenModel

TOP Module :

```
import uvm_pkg::*;
`include "uvm_macros.svh"
import shift_reg_test_pkg::*;
import ALSU_test_pkg::*;
import shared_pkg::*;
module top();
bit clk ;
always #1 clk = !clk ;
ALSU_interface ALSU_if (clk);
shift_reg_if shift_regif ();
shift_reg shiftreg (shift_regif);
ALSU  alsu (ALSU_if);

assign shift_regif.datain = ALSU_if.out;
assign ALSU_if.out_shift_reg = shift_regif.dataout ;
assign shift_regif.serial_in = alsu.serial_in_reg;
assign shift_regif.direction = direction_e'(alsu.direction_reg);
assign shift_regif.mode = mode_e'(alsu.opcode_reg[0]) ;

bind ALSU ALSU_Assertions my_assertions (ALSU_if);

initial begin
  uvm_config_db#(virtual shift_reg_if)::set(null , "uvm_test_top" ,
"shift_reg_IF" , shift_regif );
  uvm_config_db#(virtual ALSU_interface)::set(null , "uvm_test_top" ,"ALSU_if" ,
ALSU_if );

  run_test("ALSU_test");
end
endmodule
```

ALSU_Test :

```
package ALSU_test_pkg;
import uvm_pkg::*;
import ALSU_env_pkg::*;
import shift_reg_env_pkg::*;
import ALSU_seq_item_pkg::*;
import ALSU_main_seq_pkg::*;
import ALSU_last_seq_pkg::*;
import ALSU_rst_seq_pkg::*;
import ALSU_config_pkg::*;
import shift_reg_config_pkg::*;

`include "uvm_macros.svh"
class ALSU_test extends uvm_test;
`uvm_component_utils(ALSU_test)

ALSU_env ALSU_environment ;
alsu_config_obj alsu_config_obj_test ;
shift_reg_env shift_env ;
shift_reg_config shift_confg ;
ALSU_rst_seq  rst_seq  ;
ALSU_main_seq main_seq ;
ALSU_last_seq last_seq ;

function new(string name ="ALSU_test" , uvm_component parent = null);
    super.new(name , parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    alsu_config_obj_test = alsu_config_obj::type_id::create("alsu_config_obj_test");
    shift_confg = shift_reg_config::type_id::create("shift_confg");

    if(! uvm_config_db#(virtual ALSU_interface)::get(this , "" , "ALSU_if" ,
alsu_config_obj_test.alsu_config_vif))
    `uvm_fatal("build phase" , "unable to get virtual interface")
    alsu_config_obj_test.is_active = UVM_ACTIVE;
    uvm_config_db#(alsu_config_obj)::set(this , "*" , "ALSU_CFG" , alsu_config_obj_test );
    if (! uvm_config_db#(virtual shift_reg_if)::get(this ,"" , "shift_reg_IF" ,
shift_confg.config_if))
    `uvm_fatal("build_phase" , "unabe to get virtual interface")
    shift_confg.is_active = UVM_PASSIVE ;
    uvm_config_db#(shift_reg_config)::set(this ,"*" , "SHIFTREG_CFG" , shift_confg);
```

```
    ALSU_environment = ALSU_env::type_id::create("ALSU_env" , this);
    shift_env = shift_reg_env::type_id::create("shift_reg_env" , this);

    rst_seq  = ALSU_rst_seq::type_id::create("rst_seq")   ;
    main_seq = ALSU_main_seq::type_id::create("main_seq") ;
    last_seq = ALSU_last_seq::type_id::create("last_seq") ;

endfunction


task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
    `uvm_info("run_phase" , "Reset Assereted" , UVM_LOW);
    rst_seq.start(ALSU_environment.agent.sequencer);
    `uvm_info("run_phase" , "Reset Deassereted" , UVM_LOW);
    `uvm_info("run_phase" , "Stimulus generation started" , UVM_LOW);
    main_seq.start(ALSU_environment.agent.sequencer);
    `uvm_info("run_phase" , "Stimulus generation ended " , UVM_LOW);
    `uvm_info("run_phase" , "Last generation started" , UVM_LOW);
    last_seq.start(ALSU_environment.agent.sequencer);
    `uvm_info("run_phase" , "Last generation ended " , UVM_LOW);
    phase.drop_objection(this);

  endtask

endclass
endpackage
```

ALSU FILES :

DESIGN :

```
import ALSU_pkg::*;
module ALSU(ALSU_interface.DUT ALSU_if);

reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
reg signed [1:0] cin_reg;
reg [2:0]  opcode_reg;
reg signed [2:0] A_reg, B_reg;
wire invalid_red_op, invalid_opcode, invalid;

//Invalid handling
assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
assign invalid = invalid_red_op | invalid_opcode;

//Registering input signals
always @(posedge ALSU_if.clk or posedge ALSU_if.rst) begin
  if(ALSU_if.rst) begin
     cin_reg       <= 0;
     red_op_B_reg  <= 0;
     red_op_A_reg  <= 0;
     bypass_B_reg  <= 0;
     bypass_A_reg  <= 0;
     direction_reg <= 0;
     serial_in_reg <= 0;
     opcode_reg    <= 0 ;
     A_reg <= 0;
     B_reg <= 0;
  end else begin
     cin_reg       <= ALSU_if.cin       ;
     red_op_B_reg  <= ALSU_if.red_op_B  ;
     red_op_A_reg  <= ALSU_if.red_op_A  ;
     bypass_B_reg  <= ALSU_if.bypass_B  ;
     bypass_A_reg  <= ALSU_if.bypass_A  ;
     direction_reg <= ALSU_if.direction ;
     serial_in_reg <= ALSU_if.serial_in ;
     opcode_reg    <= ALSU_if.opcode    ;
     A_reg         <= ALSU_if.A         ;
     B_reg         <= ALSU_if.B         ;
```

```
  end
end

//leds output blinking
always @(posedge ALSU_if.clk or posedge ALSU_if.rst) begin
  if(ALSU_if.rst) begin
     ALSU_if.leds <= 0;
  end else begin
      if (invalid)
        ALSU_if.leds <= ~ALSU_if.leds;
      else
        ALSU_if.leds <= 0;
  end
end

//ALSU output processing
always @(posedge ALSU_if.clk or posedge ALSU_if.rst) begin
  if(ALSU_if.rst) begin
    ALSU_if.out <= 0;
  end
  else begin
    if (bypass_A_reg && bypass_B_reg)
      ALSU_if.out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
    else if (bypass_A_reg)
      ALSU_if.out <= A_reg;
    else if (bypass_B_reg)
      ALSU_if.out <= B_reg;
    else if (invalid)
      ALSU_if.out <= 0;
    else begin
        case (opcode_reg)
           OR: begin
            if (red_op_A_reg && red_op_B_reg)
              ALSU_if.out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
            else if (red_op_A_reg)
              ALSU_if.out <= |A_reg;
            else if (red_op_B_reg)
              ALSU_if.out <= |B_reg;
            else
              ALSU_if.out <= A_reg | B_reg;
          end
          XOR: begin
            if (red_op_A_reg && red_op_B_reg)
              ALSU_if.out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
            else if (red_op_A_reg)
```

```
              ALSU_if.out <= ^A_reg;
            else if (red_op_B_reg)
              ALSU_if.out <= ^B_reg;
            else
              ALSU_if.out <= A_reg ^ B_reg;
          end
          ADD: if (FULL_ADDER == "ON") begin
                    ALSU_if.out <= A_reg + B_reg + cin_reg ;
                   end else begin
                    ALSU_if.out <= A_reg + B_reg;
                   end
          MULT:   ALSU_if.out <= A_reg * B_reg ;
          SHIFT:  ALSU_if.out <= ALSU_if.out_shift_reg ;
          ROTATE: ALSU_if.out <= ALSU_if.out_shift_reg ;
        endcase
    end
  end
end

endmodule
```

ALSU_interface :

```
import ALSU_pkg::*;
import shared_pkg::*;
interface ALSU_interface (clk);

input clk ;
logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, serial_in, direction;
logic[2:0] opcode;
logic signed [2:0] A, B;
logic [15:0] leds;
logic signed [5:0] out ;
logic [5:0] out_shift_reg ;

modport DUT (
        input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction,
serial_in, opcode, A, B , out_shift_reg ,
        output leds, out
    );

endinterface
```

ALSU_pkg:

```
package ALSU_pkg;
    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";
    typedef enum bit [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6,
INVALID_7} opcode_e;
    typedef enum bit [2:0] {MAXPOS=3'b011, ZERO=3'b000, MAXNEG=3'b100} reg_e;
endpackage
```

ALSU_env :

```
package ALSU_env_pkg;
import ALSU_agent_pkg::*;
import ALSU_scoreboard_pkg::*;
import ALSU_coverage_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
    class ALSU_env extends uvm_env;
    `uvm_component_utils(ALSU_env);
    ALSU_agent agent ;
    ALSU_scoreboard scoreboard;
    ALSU_coverage coverage ;

    function new (string name = "ALSU_env" , uvm_component parent = null);
        super.new(name , parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agent      = ALSU_agent::type_id::create("agent" , this);
        scoreboard = ALSU_scoreboard::type_id::create("scoreboard" , this);
        coverage   = ALSU_coverage::type_id::create("coverage" , this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agent.agent_ap.connect(scoreboard.sb_export);
        agent.agent_ap.connect(coverage.cov_export);
    endfunction
    endclass
endpackage
```

ALSU_scoreboard :

```
package ALSU_scoreboard_pkg;
import uvm_pkg::*;
import ALSU_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"

class ALSU_scoreboard extends uvm_scoreboard;
`uvm_component_utils(ALSU_scoreboard)
uvm_analysis_export #(ALSU_seq_item) sb_export ;
uvm_tlm_analysis_fifo #(ALSU_seq_item) sb_fifo ;
ALSU_seq_item seq_item_sb;

bit clk, rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
bit signed [2:0] A, B;
logic [2:0] opcode;
bit [15:0] leds_exp;
logic [15:0] leds;
logic signed [5:0] out;
bit signed [5:0] out_exp;

// Internals to register the stimulus for golden model
bit red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg,
serial_in_reg;
bit signed [1:0] cin_reg;
bit [2:0] opcode_reg;
bit signed [2:0] A_reg, B_reg;

int error_count = 0;
int correct_count = 0;

    function new(string name ="ALSU_scoreboard" , uvm_component parent = null);
        super.new(name , parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export" , this);
        sb_fifo = new("sb_fifo" , this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
```

```
        sb_export.connect(sb_fifo.analysis_export) ;
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
          sb_fifo.get(seq_item_sb)          ;
          cin       = seq_item_sb.cin       ;
          red_op_B  = seq_item_sb.red_op_B  ;
          red_op_A  = seq_item_sb.red_op_A  ;
          bypass_B  = seq_item_sb.bypass_B  ;
          bypass_A  = seq_item_sb.bypass_A  ;
          direction = seq_item_sb.direction ;
          serial_in = seq_item_sb.serial_in ;
          opcode    = seq_item_sb.opcode    ;
          A         = seq_item_sb.A         ;
          B         = seq_item_sb.B         ;
          if(seq_item_sb.rst)begin
            check_reset();
            reset_internals();
          end else begin
            reference_model();
            check_results();
          end
        end
    endtask

    task check_results();
      if(out_exp !== seq_item_sb.out || leds_exp !== seq_item_sb.leds)begin
          `uvm_error("run_phase" , $sformatf("Transaction failed : Transaction
received by DUT : %s while the reference out: 0b:%b , reference leds = %b" ,
seq_item_sb.convert2string , out_exp , leds_exp))
          error_count++ ;
          $stop ;
      end
      else begin
          `uvm_info("run_phase"  , $sformatf("Correct dataout: Transaction
received by DUT : %s while the reference out: 0b:%b , reference leds = %b" ,
seq_item_sb.convert2string , out_exp , leds_exp) , UVM_HIGH)
          correct_count++ ;
        end
    endtask

    task check_reset();
      if(seq_item_sb.out !== 0 || seq_item_sb.leds !== 0 )begin
```

```
      `uvm_error("run_phase" , $sformatf("Transaction failed : Transaction
received by DUT : %s while the reference out: 0b:%b , reference leds = %b" ,
seq_item_sb.convert2string , out_exp , leds_exp))
          error_count++ ;
      end
      else begin
          `uvm_info("run_phase"  , $sformatf("Correct dataout: Transaction
received by DUT : %s while the reference out: 0b:%b , reference leds = %b" ,
seq_item_sb.convert2string , out_exp , leds_exp) , UVM_HIGH)
          correct_count++ ;
          end
    endtask

    function bit is_invalid();
      // invalid case 1
      if (opcode_reg == INVALID_6 || opcode_reg == INVALID_7)
          return 1;
      // invalid case 2
      else if ((opcode_reg > 3'b001) && (red_op_A_reg || red_op_B_reg))
          return 1;
      else
          return 0;
    endfunction

    task reset_internals();
      {red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg,
serial_in_reg} = 6'b000000;
      cin_reg = 2'b00;
      opcode_reg = 3'b000;
      {A_reg, B_reg} = 2'b00;
    endtask

    task reference_model();
    // Handle leds_exp
    if (is_invalid())
        leds_exp = ~leds_exp;
    else
        leds_exp = 0;

    // Check Bypass
    if (bypass_A_reg)
        out_exp = A_reg;
    else if (bypass_B_reg)
        out_exp = B_reg;
```

```
    // Check invalid
    else if (is_invalid())
        out_exp = 0;

    // Valid opcodes
    else begin
        // OR
        if (opcode_reg == OR) begin
            if (red_op_A_reg)
                out_exp = |A_reg;
            else if (red_op_B_reg)
                out_exp = |B_reg;
            else
                out_exp = A_reg | B_reg;
        end

        // XOR
        else if (opcode_reg == XOR) begin
            if (red_op_A_reg)
                out_exp = ^A_reg;
            else if (red_op_B_reg)
                out_exp = ^B_reg;
            else
                out_exp = A_reg ^ B_reg;
        end

        // ADD
        else if (opcode_reg == ADD)
            out_exp = A_reg + B_reg + cin_reg;

        // MULT
        else if (opcode_reg == MULT)
            out_exp = A_reg * B_reg;

        // SHIFT
        else if (opcode_reg == SHIFT) begin
            if (direction_reg)
                out_exp = {out_exp[4:0], serial_in_reg};
            else
                out_exp = {serial_in_reg, out_exp[5:1]};
        end

        // ROTATE
        else if (opcode_reg == ROTATE) begin
            if (direction_reg)
```

```
                out_exp = {out_exp[4:0], out_exp[5]};
            else
                out_exp = {out_exp[0], out_exp[5:1]};
        end
    end
    update_internals();
endtask

task update_internals();
    cin_reg       = cin       ;
    red_op_B_reg  = red_op_B  ;
    red_op_A_reg  = red_op_A  ;
    bypass_B_reg  = bypass_B  ;
    bypass_A_reg  = bypass_A  ;
    direction_reg = direction ;
    serial_in_reg = serial_in ;
    opcode_reg    = opcode    ;
    A_reg         = A         ;
    B_reg         = B         ;
endtask

function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        `uvm_info("run_phase"  , $sformatf("Total Successful Transactions = %0d"
,correct_count) , UVM_MEDIUM);
        `uvm_info("run_phase"  , $sformatf("Total Failed  Transactions = %0d" ,
error_count ) , UVM_MEDIUM);
endfunction


endclass

endpackage
```

ALSU_Coverage :

```
package ALSU_coverage_pkg;
import uvm_pkg::*;
import ALSU_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"
    class ALSU_coverage extends uvm_component;
    `uvm_component_utils(ALSU_coverage);
    uvm_analysis_export #(ALSU_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(ALSU_seq_item) cov_fifo;
    ALSU_seq_item cov_seq_item;

    covergroup cvr_gp ;
    A_cp : coverpoint cov_seq_item.A {
        bins A_data_0 = {0};
        bins A_data_max = {MAXPOS};
        bins A_data_min = {MAXNEG};
        bins A_data_default = default ;
        }


    A_data_walkingones_cp : coverpoint cov_seq_item.A iff(cov_seq_item.red_op_A){
        bins A_data_walkingones[] ={3'b001, 3'b010, 3'b100};
    }

    B_cp : coverpoint cov_seq_item.B {
        bins B_data_0 = {0};
        bins B_data_max = {MAXPOS};
        bins B_data_min = {MAXNEG};
        bins B_data_default = default ;
        }


     B_data_walkingones_cp : coverpoint cov_seq_item.B iff(!cov_seq_item.red_op_A &
cov_seq_item.red_op_B){
        bins B_data_walkingones[] ={3'b001, 3'b010, 3'b100};
    }


    ALU_cp : coverpoint cov_seq_item.opcode {
        bins Bins_shift  [] = {SHIFT , ROTATE};
        bins Bins_arith  [] = {ADD , MULT};
        bins Bins_bitwise[] = {OR , XOR};
```

```
    illegal_bins Bins_invalid[] = {INVALID_6 , INVALID_7};
    bins Bins_trans  [] = (0=>1=>2=>3=>4=>5);
    }
//============================ for cross covrage ==============================

C_cp : coverpoint cov_seq_item.cin {
    option.weight = 0 ;
    bins cin_0 = {0};
    bins cin_1 = {1};
}

direction_cp : coverpoint cov_seq_item.direction {
    option.weight = 0 ;
    bins dir_0 = {0};
    bins dir_1 = {1};
}

shift_cp : coverpoint cov_seq_item.serial_in {
    option.weight = 0 ;
    bins shift_in_0 = {0};
    bins shift_in_1 = {1};
}

A_Walking_ones : coverpoint cov_seq_item.A {
    option.weight = 0 ;
    bins A_1 = {3'b001};
    bins A_2 = {3'b010};
    bins A_3 = {3'b100};
}

B_Walking_ones : coverpoint cov_seq_item.B {
    option.weight = 0 ;
    bins B_1 = {3'b001};
    bins B_2 = {3'b010};
    bins B_3 = {3'b100};
}

A_zero : coverpoint cov_seq_item.A {
    option.weight = 0 ;
    bins A_zero = {3'b000} ;
}

B_zero : coverpoint cov_seq_item.B {
    option.weight = 0 ;
    bins B_zero = {3'b000} ;
```

```
    }


    red_op_A_1_cp : coverpoint cov_seq_item.red_op_A {
        option.weight = 0 ;
        bins red_op_A_1 = {1};
    }

    red_op_A_cp : coverpoint cov_seq_item.red_op_A {
        option.weight = 0 ;
        bins red_op_A_0  = {0};
        bins red_op_A_1 = {1};
    }

    red_op_B_1_cp : coverpoint cov_seq_item.red_op_B {
        option.weight = 0 ;
        bins red_op_B_1 = {1};
    }

    red_op_B_cp : coverpoint cov_seq_item.red_op_B {
        option.weight = 0 ;
        bins red_op_B_0 = {0};
        bins red_op_B_1 = {1};
    }

    ALU_ADD_MULT :coverpoint cov_seq_item.opcode {
        option.weight = 0 ;
        bins add  = {ADD}  ;
        bins mult = {MULT} ;
    }
    ALU_SHIFT_ROTATE :coverpoint cov_seq_item.opcode {
        option.weight = 0 ;
        bins shift  = {SHIFT}  ;
        bins rotate = {ROTATE} ;
    }
    ALU_OR_XOR :coverpoint cov_seq_item.opcode {
        option.weight = 0 ;
        bins my_or  = {OR}  ;
        bins my_xor = {XOR} ;
    }
    ALU_invalid_with_red : coverpoint cov_seq_item.opcode {
        option.weight = 0 ;
        bins my_or     = {OR}        ;
        bins my_xor    = {XOR}       ;
        bins add       = {ADD}       ;
```

```
        bins mult      = {MULT}      ;
        bins shift     = {SHIFT}     ;
        bins rotate    = {ROTATE}    ;
        bins invalid_6 = {INVALID_6} ;
        bins invalid_7 = {INVALID_7} ;
    }


    //1. Bins_arith with A and B corners
    CROSS_add_mult_with_A_B_Corners: cross A_cp ,B_cp ,ALU_ADD_MULT ;

    //2. addition with cin 0 or 1
    CROSS_Addition_with_cin_0_or_1 :cross ALU_ADD_MULT , C_cp {
        option.cross_auto_bin_max = 0;
        bins Addition_with_cin_0 = binsof(ALU_ADD_MULT.add)&& binsof(C_cp.cin_0);
        bins Addition_with_cin_1 = binsof(ALU_ADD_MULT.add)&& binsof(C_cp.cin_1);
    }

    //3. Shift_or_Rotate_with_dir_0_or_1
    CROSS_Shift_or_Rotate_with_dir_0_or_1 :cross ALU_SHIFT_ROTATE , direction_cp ;

    //4. Shift_with_shiftin_0_or_1
    CROSS_Shift_with_shiftin_0_or_1 :cross shift_cp , ALU_SHIFT_ROTATE {
        option.cross_auto_bin_max = 0;
        bins Shift_with_shift_0 = binsof(ALU_SHIFT_ROTATE.shift) &&
binsof(shift_cp.shift_in_0);
        bins Shift_with_shift_1 = binsof(ALU_SHIFT_ROTATE.shift) &&
binsof(shift_cp.shift_in_1);
    }
    //5. OR_XOR_with_RED_A
    CROSS_OR_XOR_with_red_A :cross ALU_OR_XOR ,A_Walking_ones , red_op_A_1_cp  ,B_zero ;

    //6. OR_XOR_with_RED_A
    CROSS_OR_XOR_with_red_B :cross ALU_OR_XOR ,B_Walking_ones , red_op_B_1_cp  ,A_zero ;

    //7. invalid_case_red_op_OR_XOR
    CROSS_invalid_case_red_op :cross  ALU_invalid_with_red , red_op_A_cp , red_op_B_cp {
       ignore_bins ignored = binsof(ALU_invalid_with_red.my_or) ||
binsof(ALU_invalid_with_red.my_xor)   ;
       ignore_bins red_invalid = (binsof(ALU_invalid_with_red.add) ||
binsof(ALU_invalid_with_red.mult) || binsof(ALU_invalid_with_red.shift) ||
binsof(ALU_invalid_with_red.rotate)) && (binsof(red_op_A_cp.red_op_A_0) &&
binsof(red_op_B_cp.red_op_B_0));
    }
    endgroup
```

```
    function new (string name = "ALSU_coverage" , uvm_component parent = null);
        super.new(name , parent);
        cvr_gp = new();
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        cov_export = new("cov_export" , this) ;
        cov_fifo   = new( "cov_fifo"  , this) ;
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        cov_export.connect(cov_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cov_fifo.get(cov_seq_item);
            cvr_gp.sample();
        end
    endtask

    endclass
endpackage
```

ALSU_agent :

```
package ALSU_agent_pkg;
import uvm_pkg::*;
import ALSU_config_pkg::*;
import ALSU_sequencer_pkg::*;
import ALSU_driver_pkg::*;
import ALSU_monitor_pkg::*;
import ALSU_seq_item_pkg::*;

`include "uvm_macros.svh"
class ALSU_agent extends uvm_agent;
`uvm_component_utils(ALSU_agent)
alsu_config_obj ALSU_confg;
ALSU_driver driver;
ALSU_monitor monitor;
ALSU_sequencer sequencer;
uvm_analysis_port #(ALSU_seq_item) agent_ap;
    function new(string name ="ALSU_agent" , uvm_component parent = null);
        super.new(name , parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db #(alsu_config_obj)::get(this , "" , "ALSU_CFG" , ALSU_confg))
        `uvm_fatal("build phase" , "unable to get configuration object")
        agent_ap = new("agent_ap", this);
        monitor = ALSU_monitor::type_id::create("monitor", this );
        if(ALSU_confg.is_active === UVM_ACTIVE)begin
            driver = ALSU_driver::type_id::create("driver", this );
            sequencer = ALSU_sequencer::type_id::create("sequencer", this );
        end
    endfunction
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        monitor.alsu_monitor_vif = ALSU_confg.alsu_config_vif;
        monitor.mon_ap.connect(agent_ap);
        if(ALSU_confg.is_active === UVM_ACTIVE)begin
            driver.alsu_driver_vif = ALSU_confg.alsu_config_vif;
            driver.seq_item_port.connect(sequencer.seq_item_export);
        end
        endfunction
endclass
endpackage
```

ALSU_sequencer :

```
package ALSU_sequencer_pkg;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"
class ALSU_sequencer extends uvm_sequencer #(ALSU_seq_item);
`uvm_component_utils(ALSU_sequencer)

function new(string name = "ALSU_sequencer" ,uvm_component parent = null );
    super.new(name , parent);
endfunction
endclass
endpackage
```

ALSU_monitor :

```
package ALSU_monitor_pkg ;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"
    class ALSU_monitor extends uvm_monitor;
        `uvm_component_utils(ALSU_monitor)

        ALSU_seq_item mon_seq_item ;
        virtual ALSU_interface alsu_monitor_vif;
        uvm_analysis_port #(ALSU_seq_item) mon_ap;

        function new(string name ="ALSU_monitor" , uvm_component parent = null);
            super.new(name , parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            mon_ap = new("mon_ap",this);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            `uvm_info("MONITOR" ,"MONITOR started",UVM_MEDIUM)

            forever begin
                mon_seq_item = ALSU_seq_item::type_id::create("seq_item");
                @(negedge alsu_monitor_vif.clk);
```

```
                mon_seq_item.rst       = alsu_monitor_vif.rst       ;
                mon_seq_item.cin       = alsu_monitor_vif.cin       ;
                mon_seq_item.red_op_B  = alsu_monitor_vif.red_op_B  ;
                mon_seq_item.red_op_A  = alsu_monitor_vif.red_op_A  ;
                mon_seq_item.bypass_B  = alsu_monitor_vif.bypass_B  ;
                mon_seq_item.bypass_A  = alsu_monitor_vif.bypass_A  ;
                mon_seq_item.direction = alsu_monitor_vif.direction ;
                mon_seq_item.serial_in = alsu_monitor_vif.serial_in ;
                mon_seq_item.opcode    = alsu_monitor_vif.opcode    ;
                mon_seq_item.A         = alsu_monitor_vif.A         ;
                mon_seq_item.B         = alsu_monitor_vif.B         ;
                mon_seq_item.out       = alsu_monitor_vif.out       ;
                mon_seq_item.leds      = alsu_monitor_vif.leds      ;
                mon_ap.write(mon_seq_item);
                `uvm_info("run_phase" ,mon_seq_item.convert2string_stimulus ,
UVM_HIGH)
            end

        endtask
    endclass
endpackage
```

ALSU_Driver :

```
package ALSU_driver_pkg ;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
import ALSU_seq_item_pkg::*;
import shared_pkg::*;
`include "uvm_macros.svh"
    class ALSU_driver extends uvm_driver #(ALSU_seq_item);
        `uvm_component_utils(ALSU_driver)

        ALSU_seq_item seq_item ;
        virtual ALSU_interface alsu_driver_vif;

        function new(string name ="ALSU_driver" , uvm_component parent = null);
            super.new(name , parent);
        endfunction


        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                seq_item = ALSU_seq_item::type_id::create("seq_item");
                seq_item_port.get_next_item(seq_item);
                alsu_driver_vif.rst       = seq_item.rst       ;
                alsu_driver_vif.cin       = seq_item.cin       ;
                alsu_driver_vif.red_op_B  = seq_item.red_op_B  ;
                alsu_driver_vif.red_op_A  = seq_item.red_op_A  ;
                alsu_driver_vif.bypass_B  = seq_item.bypass_B  ;
                alsu_driver_vif.bypass_A  = seq_item.bypass_A  ;
                alsu_driver_vif.direction = direction_e'(seq_item.direction) ;
                alsu_driver_vif.serial_in = seq_item.serial_in ;
                alsu_driver_vif.opcode    = seq_item.opcode    ;
                alsu_driver_vif.A         = seq_item.A         ;
                alsu_driver_vif.B         = seq_item.B         ;
                @(negedge alsu_driver_vif.clk);
                seq_item_port.item_done();
                `uvm_info("run_phase" ,seq_item.convert2string_stimulus ,
UVM_HIGH)
            end
        endtask
    endclass
endpackage
```

ALSU_seq_item :

```
package ALSU_seq_item_pkg;
import ALSU_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ALSU_seq_item extends uvm_sequence_item;
`uvm_object_utils(ALSU_seq_item)
    rand logic rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction,
serial_in;
    rand logic [2:0] A, B;
    rand logic[2:0] opcode;
    logic [15:0] leds;
    logic signed [5:0] out;
    rand reg_e enum_ext_A, enum_ext_B;
    rand logic [3:0] A_rem_values, B_rem_values;
    rand bit [2:0] walking_ones[] = '{3'b001, 3'b010, 3'b100};
    rand bit [2:0] walking_ones_t, walking_ones_f;
    rand opcode_e opcode_valid [6];

    // ALSU_1
    constraint rst_con {
        rst dist {1:= 2, 0:= 98};
    }

    constraint A_B_con {
        A_rem_values != MAXPOS || 0 || MAXNEG;
        B_rem_values != MAXPOS || 0 || MAXNEG;
        walking_ones_t inside {walking_ones};
        !(walking_ones_f inside {walking_ones});

        if (opcode == OR || opcode == XOR) {
            // ALSU_5, ALSU_7
            if (red_op_A == 1) {
                B == 0;
                A dist {walking_ones_t:= 80, walking_ones_f:= 20};
            } else if (red_op_B == 1) {
                A == 0;
                B dist {walking_ones_t:= 80, walking_ones_f:= 20};
            }
        } else {
            // ALSU_4, ALSU_6
            red_op_A dist {1:= 20, 0:= 80};
            red_op_B dist {1:= 20, 0:= 80};
```

```
            // ALSU_2, ALSU_3
            if (opcode == ADD || opcode == MULT) {
                A dist {enum_ext_A:= 80, A_rem_values:= 20};
                B dist {enum_ext_B:= 80, B_rem_values:= 20};
            }
        }
    }
    // ALSU_10
    constraint opcode_con {
        opcode dist {[OR:ROTATE]:= 80, [INVALID_6:INVALID_7]:= 20};
    }
    // ALSU_11
    constraint bypass_con {
        bypass_A dist {1:= 2, 0:= 98};
        bypass_B dist {1:= 2, 0:= 98};
    }
    //ALSU_12
    constraint fixed_arr_con {
        foreach (opcode_valid[i]) {
            opcode_valid[i] inside {[OR:ROTATE]};
        }
       unique {opcode_valid};
    }

    function new(string name = "ALSU sequence item");
        super.new(name);
    endfunction

    function string convert2string();
        return $sformatf("%s , rst = 0b%b, cin = 0b%b, red_op_A= 0b%b, red_op_B=
0b%b, bypass_A= 0b%b, bypass_B= 0b%b, direction= 0b%b, serial_in= 0b%b ,opcode =
0b%b , A = 0b%b, B = 0b%b , leds = 0b%b , out == 0b%b",super.convert2string()
,rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in , opcode
, A, B , leds , out );
    endfunction

    function string convert2string_stimulus();
        return $sformatf("rst = 0b%b, cin = 0b%b, red_op_A= 0b%b, red_op_B= 0b%b,
bypass_A= 0b%b, bypass_B= 0b%b, direction= 0b%b, serial_in= 0b%b ,opcode = 0b%b ,
A = 0b%b, B = 0b%b",rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction,
serial_in , opcode , A, B );
    endfunction
endclass
endpackage
```

ALSU_reset_sequence :

```
package ALSU_rst_seq_pkg;
import ALSU_pkg::*;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"

class ALSU_rst_seq extends uvm_sequence #(ALSU_seq_item);
`uvm_object_utils(ALSU_rst_seq)
ALSU_seq_item rst_seq_item ;

function new(string name = "ALSU_rst_seq");
    super.new(name);
endfunction

task body();
    rst_seq_item = ALSU_seq_item::type_id::create("rst_seq_item");
    start_item(rst_seq_item);
    rst_seq_item.rst        = 1 ;
    rst_seq_item.cin        = 0 ;
    rst_seq_item.red_op_B   = 0 ;
    rst_seq_item.red_op_A   = 0 ;
    rst_seq_item.bypass_B   = 0 ;
    rst_seq_item.bypass_A   = 0 ;
    rst_seq_item.direction  = 0 ;
    rst_seq_item.serial_in  = 0 ;
    rst_seq_item.opcode     = opcode_e'(0) ;
    rst_seq_item.A          = 0 ;
    rst_seq_item.B          = 0 ;
    finish_item(rst_seq_item);
endtask


endclass

endpackage
```

ALSU_main_sequence :

```
package ALSU_main_seq_pkg;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"

class ALSU_main_seq extends uvm_sequence #(ALSU_seq_item);
`uvm_object_utils(ALSU_main_seq)
ALSU_seq_item main_seq_item ;

function new(string name = "ALSU_main_seq");
    super.new(name);
endfunction

task body();
    repeat(500000) begin
        main_seq_item = ALSU_seq_item::type_id::create("main_seq_item");
        start_item(main_seq_item);
        assert(main_seq_item.randomize());
        finish_item(main_seq_item);
    end
endtask
endclass
endpackage
```

ALSU_Last_sequence :

```
package ALSU_last_seq_pkg;
import uvm_pkg::*;
import ALSU_seq_item_pkg::*;
`include "uvm_macros.svh"

class ALSU_last_seq extends uvm_sequence #(ALSU_seq_item);
`uvm_object_utils(ALSU_last_seq)
ALSU_seq_item last_seq ;

function new(string name = "ALSU_last_seq");
    super.new(name);
endfunction
```

```
task body();
        repeat(100) begin
        last_seq = ALSU_seq_item::type_id::create("last_seq_item");

        last_seq.constraint_mode(0);
        last_seq.fixed_arr_con.constraint_mode(1);
        last_seq.rst_con.constraint_mode(1);
        assert(last_seq.randomize());

        foreach(last_seq.opcode_valid[i]) begin
                start_item(last_seq);
                last_seq.opcode = last_seq.opcode_valid[i];
                finish_item(last_seq);
            end
        end
endtask
endclass
endpackage
```

ALSU_configuration_Object :

```
package ALSU_config_pkg ;
import uvm_pkg::*;
`include "uvm_macros.svh"
    class alsu_config_obj extends uvm_object;
    `uvm_object_utils(alsu_config_obj);

    virtual ALSU_interface alsu_config_vif ;
    uvm_active_passive_enum is_active ;

    function new(string name = "ALSU_confg");
        super.new(name);
    endfunction
    endclass
endpackage
```

ALSU_Assertions :

```
import ALSU_pkg::*;
module ALSU_Assertions(ALSU_interface.DUT ALSU_if);

wire invalid_red_op, invalid_opcode, invalid;
logic signed [1:0] mycin;

// for invalid
assign invalid_red_op = (ALSU_if.red_op_A | ALSU_if.red_op_B) &
(ALSU_if.opcode[1] | ALSU_if.opcode[2]) ;
assign invalid_opcode = (ALSU_if.opcode[1] & ALSU_if.opcode[2]) ;
assign invalid = invalid_red_op | invalid_opcode ;
assign mycin = ALSU_if.cin ;

// reset
always_comb begin
    if(ALSU_if.rst)
    rst_assert : assert final(ALSU_if.out === 0 && ALSU_if.leds === 0 )
    else $error("Error in Reset ");
end


// Bypass A
property Bypass_A_prop;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ALSU_if.bypass_A |-> ##2 ALSU_if.out === $past(ALSU_if.A,2);
endproperty


// Bypass B
property Bypass_B_prop;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    (!ALSU_if.bypass_A && ALSU_if.bypass_B) |-> ##2 ALSU_if.out ===
$past(ALSU_if.B,2) ;
endproperty

// invalid
property invalid_prop;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    invalid |-> ##2 (ALSU_if.out === 0 &&  ALSU_if.leds ===
($past(ALSU_if.leds)==16'hffff)?16'h0 : 16'hffff);
endproperty
```

```
// opcode_OR_red_A
property opcode_OR_red_A;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode) === OR && !(ALSU_if.bypass_A | ALSU_if.bypass_B)&&
ALSU_if.red_op_A) |-> ##2 ALSU_if.out === |$past(ALSU_if.A,2) ;
endproperty

// opcode_OR_red_B
property opcode_OR_red_B;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst )
    ((ALSU_if.opcode === OR)&&!(ALSU_if.bypass_A | ALSU_if.bypass_B) &&
!ALSU_if.red_op_A && ALSU_if.red_op_B) |-> ##2 ALSU_if.out ===
|$past(ALSU_if.B,2) ;
endproperty

// opcode_OR
property opcode_OR;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst )
    ((ALSU_if.opcode === OR )&& !(ALSU_if.bypass_A | ALSU_if.bypass_B)&&
!ALSU_if.red_op_A && !ALSU_if.red_op_B) |-> ##2 ALSU_if.out ===
($past(ALSU_if.A,2) | $past(ALSU_if.B,2)) ;
endproperty

// opcode_XOR_red_A
property opcode_XOR_red_A;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode) === XOR && !(ALSU_if.bypass_A | ALSU_if.bypass_B)&&
ALSU_if.red_op_A) |-> ##2 ALSU_if.out === ^$past(ALSU_if.A,2) ;
endproperty

// opcode_XOR_red_B
property opcode_XOR_red_B;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode === XOR)&&!(ALSU_if.bypass_A | ALSU_if.bypass_B) &&
!ALSU_if.red_op_A && ALSU_if.red_op_B) |-> ##2 ALSU_if.out ===
^$past(ALSU_if.B,2) ;
endproperty

// opcode_XOR
property opcode_XOR;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode === XOR )&& !(ALSU_if.bypass_A | ALSU_if.bypass_B)&&
!ALSU_if.red_op_A && !ALSU_if.red_op_B) |-> ##2 ALSU_if.out ===
($past(ALSU_if.A,2) ^ $past(ALSU_if.B,2)) ;
endproperty
```

```
// ADD
property Add;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode === ADD )&& !(ALSU_if.red_op_A |ALSU_if.red_op_B) &&
!(ALSU_if.bypass_A | ALSU_if.bypass_B)) |-> ##2 ALSU_if.out === (
$past(ALSU_if.A,2) + $past(ALSU_if.B,2) + $past(mycin,2)) ;
endproperty

// Mult
property Mult;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode === MULT) && !(ALSU_if.red_op_A |ALSU_if.red_op_B) &&
!(ALSU_if.bypass_A | ALSU_if.bypass_B)) |-> ##2 ALSU_if.out ===
($past(ALSU_if.A,2) * $past(ALSU_if.B,2)) ;
endproperty

// shift left
property Shift_left;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode === SHIFT) && !(ALSU_if.red_op_A |ALSU_if.red_op_B) &&
!(ALSU_if.bypass_A | ALSU_if.bypass_B) && ALSU_if.direction) |-> ##2 ALSU_if.out
=== {$past(ALSU_if.out[4:0]) , $past(ALSU_if.serial_in , 2)} ;
endproperty

// shift right
property Shift_right;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode === SHIFT) && !(ALSU_if.red_op_A |ALSU_if.red_op_B) &&
!(ALSU_if.bypass_A | ALSU_if.bypass_B) && !ALSU_if.direction) |-> ##2 ALSU_if.out
=== {$past(ALSU_if.serial_in,2) , $past(ALSU_if.out[5:1])} ;
endproperty

// rotate left
property rotate_left;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
    ((ALSU_if.opcode === ROTATE) && !(ALSU_if.red_op_A |ALSU_if.red_op_B) &&
!(ALSU_if.bypass_A | ALSU_if.bypass_B) && ALSU_if.direction) |-> ##2 ALSU_if.out
=== {$past(ALSU_if.out[4:0]) , $past(ALSU_if.out[5])} ;
endproperty

// rotate right
property rotate_right;
    @(posedge ALSU_if.clk) disable iff(ALSU_if.rst)
```

```
    ((ALSU_if.opcode === ROTATE) && !(ALSU_if.red_op_A |ALSU_if.red_op_B) &&
!(ALSU_if.bypass_A | ALSU_if.bypass_B) && !ALSU_if.direction) |-> ##2 ALSU_if.out
=== {$past(ALSU_if.out[0]) , $past(ALSU_if.out[5:1])} ;
endproperty




Bypass_A_Assert : assert property (Bypass_A_prop)
                  else $error("Error in Bypass A");
Bypass_B_Assert : assert property (Bypass_B_prop)
                  else $error("Error in Bypass B");
invalid_Assert  : assert property (invalid_prop)
                  else $error("Error in invalid");
opcode_OR_red_A_Assert  : assert property (opcode_OR_red_A)
                  else $error("Error in opcode_OR_red_A");
opcode_OR_red_B_Assert  : assert property (opcode_OR_red_B)
                  else $error("Error in opcode_OR_red_B");
opcode_OR_Assert  : assert property (opcode_OR)
                  else $error("Error in opcode_OR");
opcode_XOR_red_A_Assert  : assert property (opcode_XOR_red_A)
                  else $error("Error in opcode_XOR_red_A");
opcode_XOR_red_B_Assert  : assert property (opcode_XOR_red_B)
                  else $error("Error in opcode_XOR_red_B");
opcode_ADD_Asset   : assert property (Add)
                  else  $error("Error in opcode_ADD");

opcode_MULT_Asset  : assert property (Mult)
                  else $error("Error in opcode_MULT");
opcode_SHIFT_left_Asset  : assert property (Shift_left)
                  else $error("Error in SHIFT_left");
opcode_SHIFT_right_Asset : assert property (Shift_right)
                  else $error("Error in Shift_right");
opcode_ROTATE_left_Asset  : assert property (rotate_left)
                  else $error("Error in rotate_left");
opcode_ROTATE_right_Asset : assert property (rotate_right)
                  else $error("Error in rotate_right");

Bypass_A_cover            : cover property (Bypass_A_prop);
Bypass_B_cover            : cover property (Bypass_B_prop);
invalid_cover             : cover property (invalid_prop) ;
opcode_OR_red_A_cover     : cover property (opcode_OR_red_A) ;
opcode_OR_red_B_cover     : cover property (opcode_OR_red_B) ;
opcode_OR_cover           : cover property (opcode_OR) ;
```

```
opcode_XOR_red_A_cover    : cover property (opcode_XOR_red_A) ;
opcode_XOR_red_B_cover    : cover property (opcode_XOR_red_B) ;
opcode_XOR_cover          : cover property (opcode_XOR) ;
opcode_ADD_cover          : cover property (Add) ;
opcode_MULT_cover         : cover property (Mult) ;
opcode_SHIFT_left_cover   : cover property (Shift_left) ;
opcode_SHIFT_right_cover  : cover property (Shift_right) ;
opcode_ROTATE_left_cover  : cover property (rotate_left) ;
opcode_ROTATE_right_cover : cover property (rotate_right) ;
endmodule
```

Shift_reg FILES :

Shift_reg Design :

```
import shared_pkg::*;
module shift_reg (shift_reg_if.DUT shift_regif );


always @(*) begin
      if (shift_regif.mode == ROTATE) // rotate
         if (shift_regif.direction == LEFT) // left
            shift_regif.dataout = {shift_regif.datain[4:0],
shift_regif.datain[5]};
         else
            shift_regif.dataout = {shift_regif.datain[0],
shift_regif.datain[5:1]};
      else // shift
         if (shift_regif.direction == LEFT) // left
            shift_regif.dataout = {shift_regif.datain[4:0],
shift_regif.serial_in};
         else
            shift_regif.dataout = {shift_regif.serial_in,
shift_regif.datain[5:1]};
end
endmodule
```

Shift_reg Package :

```
package shared_pkg;
    typedef enum {SHIFT , ROTATE} mode_e   ;
    typedef enum {RIGHT , LEFT} direction_e;
endpackage
```

Shift_reg interface :

```
import shared_pkg::*;
interface shift_reg_if ();
  logic serial_in;
  direction_e direction ;
  mode_e mode;
  logic [5:0] datain, dataout;

  modport DUT (input serial_in, direction, mode , datain, output dataout );

endinterface : shift_reg_if
```

Shift_reg environment :

```
package shift_reg_env_pkg;
import shift_reg_agent_pkg::*;
import shift_reg_scoreboard_pkg::*;
import shift_reg_coverage_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class shift_reg_env extends uvm_env;
  `uvm_component_utils(shift_reg_env)
  shift_reg_agent agent ;
  shift_reg_scoreboard scoreboard ;
  shift_reg_coverage coverage ;

  function new (string name = "shift_reg_env" , uvm_component parent = null);
      super.new(name , parent);
  endfunction

  function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      agent      = shift_reg_agent::type_id::create("shift_reg_agent",this) ;
      scoreboard =
shift_reg_scoreboard::type_id::create("shift_reg_scoreboard",this);
      coverage   =
shift_reg_coverage::type_id::create("shift_reg_coverage",this);
  endfunction

  function void connect_phase(uvm_phase phase);
      super.connect_phase(phase);
      agent.agt_ap.connect(scoreboard.sb_export);
      agent.agt_ap.connect(coverage.cov_export);
  endfunction


endclass
endpackage
```

Shift_reg_Scoreboard :

```
package shift_reg_scoreboard_pkg ;
import uvm_pkg::*;
import shift_reg_seq_item_pkg::*;
`include "uvm_macros.svh"

class shift_reg_scoreboard extends uvm_scoreboard;
`uvm_component_utils(shift_reg_scoreboard)
uvm_analysis_export #(shift_reg_seq_item) sb_export ;
uvm_tlm_analysis_fifo #(shift_reg_seq_item) sb_fifo ;
shift_reg_seq_item seq_item_sb ;
int correct_count = 0 ;
int error_count   = 0 ;
logic [5:0] dataout_ref;

    function new (string name = "shift_reg_scoreboard" , uvm_component parent =
null);
        super.new(name , parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export" , this) ;
        sb_fifo   = new( "sb_fifo"  , this) ;
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            sb_fifo.get(seq_item_sb);
            reference_model(seq_item_sb);
            if(seq_item_sb.dataout !== dataout_ref )begin
                `uvm_error("run_phase" , $sformatf("Transaction failed :
Transaction received by DUT : %s while the reference out: 0b:%b" ,
seq_item_sb.convert2string , dataout_ref))
                error_count++ ;
                $stop ;
            end
```

```
            else begin
                `uvm_info("run_phase"  , $sformatf("Correct dataout: Transaction
received by DUT : %s while the reference out: 0b:%b" , seq_item_sb.convert2string
, dataout_ref) , UVM_HIGH)
                correct_count++ ;
            end
        end
    endtask

    task reference_model(input shift_reg_seq_item ref_seq_item);
        if (ref_seq_item.reset)
       dataout_ref = 0;
   else
      if (ref_seq_item.mode) // rotate
         if (ref_seq_item.direction) // left
            dataout_ref = {ref_seq_item.datain[4:0], ref_seq_item.datain[5]};
         else
            dataout_ref = {ref_seq_item.datain[0], ref_seq_item.datain[5:1]};
      else // shift
         if (ref_seq_item.direction) // left
            dataout_ref = {ref_seq_item.datain[4:0], ref_seq_item.serial_in};
         else
            dataout_ref = {ref_seq_item.serial_in, ref_seq_item.datain[5:1]};
    endtask

    function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        `uvm_info("run_phase"  , $sformatf("Total Successful Transactions = %0d"
,correct_count) , UVM_MEDIUM);
        `uvm_info("run_phase"  , $sformatf("Total Failed  Transactions = %0d" ,
error_count ) , UVM_MEDIUM);
    endfunction
endclass

endpackage
```

Shift_reg Coverage :

```
package shift_reg_coverage_pkg;
import uvm_pkg::*;
import shift_reg_seq_item_pkg::*;
`include "uvm_macros.svh"
class shift_reg_coverage extends uvm_component;
`uvm_component_utils(shift_reg_coverage)
uvm_analysis_export #(shift_reg_seq_item) cov_export ;
uvm_tlm_analysis_fifo #(shift_reg_seq_item) cov_fifo ;
shift_reg_seq_item seq_item_cov ;
    covergroup cov_gp;
        serial_in_cov : coverpoint seq_item_cov.serial_in ;
        direction_cov : coverpoint seq_item_cov.direction ;
        mode_cov      : coverpoint seq_item_cov.mode      ;
        datain_cov    : coverpoint seq_item_cov.datain    ;
        dataout_cov   : coverpoint seq_item_cov.dataout   ;
    endgroup

    function new (string name = "shift_reg_coverage" , uvm_component parent =
null);
        super.new(name , parent);
        cov_gp = new();
    endfunction

    function void build_phase (uvm_phase phase);
        super.build_phase(phase);
        cov_export = new("cov_export" , this) ;
        cov_fifo   = new("cov_fifo"   , this) ;
    endfunction

    function void connect_phase (uvm_phase phase);
        super.connect_phase(phase);
        cov_export.connect(cov_fifo.analysis_export);
    endfunction
    task run_phase (uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cov_fifo.get(seq_item_cov);
            cov_gp.sample();
        end
    endtask
endclass
endpackage
```

Shift_reg_Agent :

```
package shift_reg_agent_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
import shift_reg_config_pkg::*;
import shift_reg_sequencer_pkg::*;
import shift_reg_monitor_pkg::*;
import shift_reg_driver_pkg::*;
import shift_reg_seq_item_pkg::*;

class shift_reg_agent extends uvm_agent ;
`uvm_component_utils(shift_reg_agent)
    shift_reg_config    shift_config     ;
    shift_reg_driver    driver    ;
    shift_reg_sequencer sequencer ;
    shift_reg_monitor   monitor   ;
    uvm_analysis_port #(shift_reg_seq_item) agt_ap;

    function new (string name = "shift_reg_sequencer" , uvm_component parent = null);
        super.new(name , parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(! uvm_config_db#(shift_reg_config)::get(this , "" , "SHIFTREG_CFG" , shift_config))
        `uvm_fatal("build_phase" ,"unable to get configuration object");
        if(shift_config.is_active === UVM_ACTIVE)begin
            driver    = shift_reg_driver::type_id::create("shift_reg_driver",this);
            sequencer = shift_reg_sequencer::type_id::create("shift_reg_sequencer",this);
        end
        monitor   = shift_reg_monitor::type_id::create("shift_reg_monitor",this);
        agt_ap    = new("agent_ap" , this);
    endfunction
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        if(shift_config.is_active === UVM_ACTIVE)begin
            driver.shift_if = shift_config.config_if;
            driver.seq_item_port.connect(sequencer.seq_item_export);
        end
        monitor.shift_if = shift_config.config_if;
        monitor.mon_ap.connect(agt_ap);
    endfunction
endclass
endpackage
```

Shift_reg_Sequencer :

```
package shift_reg_sequencer_pkg;
import uvm_pkg::*;
import shift_reg_seq_item_pkg::*;
`include "uvm_macros.svh"

class shift_reg_sequencer extends uvm_sequencer #(shift_reg_seq_item);
`uvm_component_utils(shift_reg_sequencer)

  function new (string name = "sequencer" , uvm_component parent = null);
    super.new(name , parent);
  endfunction
endclass

endpackage
```

Shift_reg_Monitor :

```
package shift_reg_monitor_pkg;
import uvm_pkg::*;
import shift_reg_seq_item_pkg::*;
`include "uvm_macros.svh"

class shift_reg_monitor extends uvm_monitor ;
    `uvm_component_utils(shift_reg_monitor)
    virtual shift_reg_if shift_if ;
    shift_reg_seq_item rsp_seq_item ;
    uvm_analysis_port #(shift_reg_seq_item) mon_ap;

    function new (string name = "monitor" , uvm_component parent = null);
        super.new(name , parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon_ap = new("mon_ap" , this);
    endfunction



    task run_phase(uvm_phase phase);
        super.run_phase(phase) ;
```

```
        forever begin
            rsp_seq_item = shift_reg_seq_item::type_id::create("rsp_seq_item");
            #2 ;
            rsp_seq_item.serial_in =  shift_if.serial_in ;
            rsp_seq_item.direction =  shift_if.direction ;
            rsp_seq_item.mode      =  shift_if.mode      ;
            rsp_seq_item.datain    =  shift_if.datain    ;
            rsp_seq_item.dataout    = shift_if.dataout  ;
            mon_ap.write(rsp_seq_item)                  ;
            `uvm_info("run_phase" ,rsp_seq_item.convert2string, UVM_HIGH);
        end
    endtask
endclass
endpackage
```

Shift_reg Driver :

```
package shift_reg_driver_pkg;
import uvm_pkg::*;
import shift_reg_seq_item_pkg::*;
`include "uvm_macros.svh"

class shift_reg_driver extends uvm_driver #(shift_reg_seq_item);
    `uvm_component_utils(shift_reg_driver)
    virtual shift_reg_if shift_if ;
    shift_reg_seq_item seq_item ;
    function new (string name = "driver" , uvm_component parent = null);
        super.new(name , parent);
    endfunction
    task run_phase(uvm_phase phase);
        super.run_phase(phase) ;
        forever begin
            seq_item = shift_reg_seq_item::type_id::create("seq_item");
            seq_item_port.get_next_item(seq_item);
            shift_if.serial_in  = seq_item.serial_in;
            shift_if.direction  = seq_item.direction;
            shift_if.mode       = seq_item.mode     ;
            shift_if.datain     = seq_item.datain   ;
            #2 ;
            seq_item_port.item_done();
            `uvm_info("run_phase" ,seq_item.convert2string_stimulus , UVM_HIGH);
        end
    endtask
endclass
endpackage
```

Shift_reg Main sequence :

```
package shift_reg_main_sequence_pkg;
import uvm_pkg::*;
import shift_reg_seq_item_pkg::*;
`include "uvm_macros.svh"
    class shift_reg_main_sequence extends uvm_sequence #(shift_reg_seq_item);
    `uvm_object_utils(shift_reg_main_sequence)
    shift_reg_seq_item seq_item ;
    function new (string name = "main_sequence" );
        super.new(name);
    endfunction
    task body();
        repeat(10000)begin
            seq_item =shift_reg_seq_item::type_id::create("seq_item");
            start_item(seq_item);
            assert(seq_item.randomize());
            finish_item(seq_item);
        end
    endtask
    endclass
endpackage
```

Shift_reg sequence item :

```
package shift_reg_seq_item_pkg;
import uvm_pkg::*;
import shared_pkg::*;
`include "uvm_macros.svh"
    class shift_reg_seq_item extends uvm_sequence_item;
    `uvm_object_utils(shift_reg_seq_item);

    rand bit reset;
    rand bit serial_in;
    rand direction_e direction ;
    rand mode_e mode;
    rand bit [5:0] datain;
    logic [5:0] dataout ;

    constraint rst_con {
        reset dist {0:=98 , 1:=2};
    }

    function new (string name = "shift_reg_seq_item" );
        super.new(name);
    endfunction

    function string convert2string();
        return $sformatf("%s , reset = 0b%b , serial in = 0b%b , direction = %0s
, mode = %0s , datain = 0b%b , dataout = 0b%b",super.convert2string() ,reset ,
serial_in , direction , mode , datain , dataout );
    endfunction

    function string convert2string_stimulus();
        return $sformatf("reset = 0b%b , serial in = 0b%b , direction = %0s ,
mode = %0s , datain = 0b%b " ,reset , serial_in , direction , mode , datain );
    endfunction

    endclass
endpackage
```

Shift_reg Configuration Object :

```
package shift_reg_config_pkg ;
import uvm_pkg::*;
`include "uvm_macros.svh"

class shift_reg_config extends uvm_object;
`uvm_object_utils(shift_reg_config)

  virtual shift_reg_if config_if ;
  uvm_active_passive_enum is_active ;

  function new (string name = "shift_reg_config");
    super.new(name);
  endfunction
endclass
endpackage
```

Do File :

```
vlib work
vlog -f src_files.list +cover -covercells
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover
add wave /top/ALSU_if/* /top/shift_regif/*
coverage save ALSU.ucdb -onexit
run -all

coverage exclude -du ALSU -togglenode cin_reg

#quit -sim
#vcover report ALSU_tb.ucdb -details -annotate -all -output ALSU_coverage.txt

```

Src_files.list :

ALSU_pkg.sv

shared_pkg.sv

shift_reg_if.sv

ALSU_if.sv

ALSU.sv

shift_reg.sv

ALSU_assertions.sv

ALSU_config.sv

shift_reg_config.sv

ALSU_seq_item.sv

shift_reg_seq_item.sv

ALSU_rst_seq.sv

ALSU_main_seq.sv

ALSU_last_seq.sv

shift_reg_main_sequence.sv

ALSU_driver.sv

shift_reg_driver.sv

ALSU_monitor.sv

shift_reg_monitor.sv

ALSU_sequencer.sv

shift_reg_sequencer.sv

ALSU_agent.sv

shift_reg_agent.sv

ALSU_coverage.sv

shift_reg_coverage.sv

ALSU_scoreboard.sv

shift_reg_scoreboard.sv

ALSU_env.sv

shift_reg_env.sv

ALSU_test.sv

top.sv

Shift_reg Coverage :

Code Coverage :

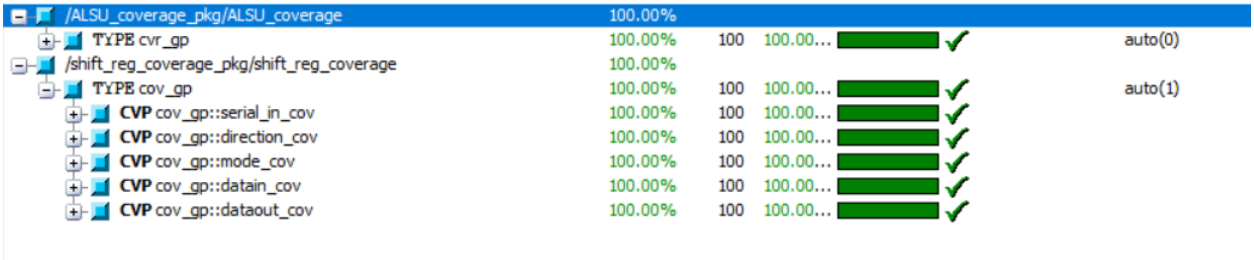
```
================================================================================
=== Instance: /top/shiftreg
=== Design Unit: work.shift_reg
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         5         5         0   100.00%
```

```
Condition Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       3         3         0   100.00%
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       5         5         0   100.00%
```

Functional Coverage :

Covergroup	Metric	Goal	Bins	Status
TYPE /shift_reg_coverage_pkg/shift_reg_coverage/cov_gp				
	100.00%	100	-	Covered
covered/total bins:	134	134	-	
missing/total bins:	0	134	-	
% Hit:	100.00%	100	-	



ALSU Coverage

Code Coverage :

```
================================================================================
=== Instance: /top/alsu
=== Design Unit: work.ALSU
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        28        27         1    96.42%
```

```
Condition Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       6         6         0   100.00%
```

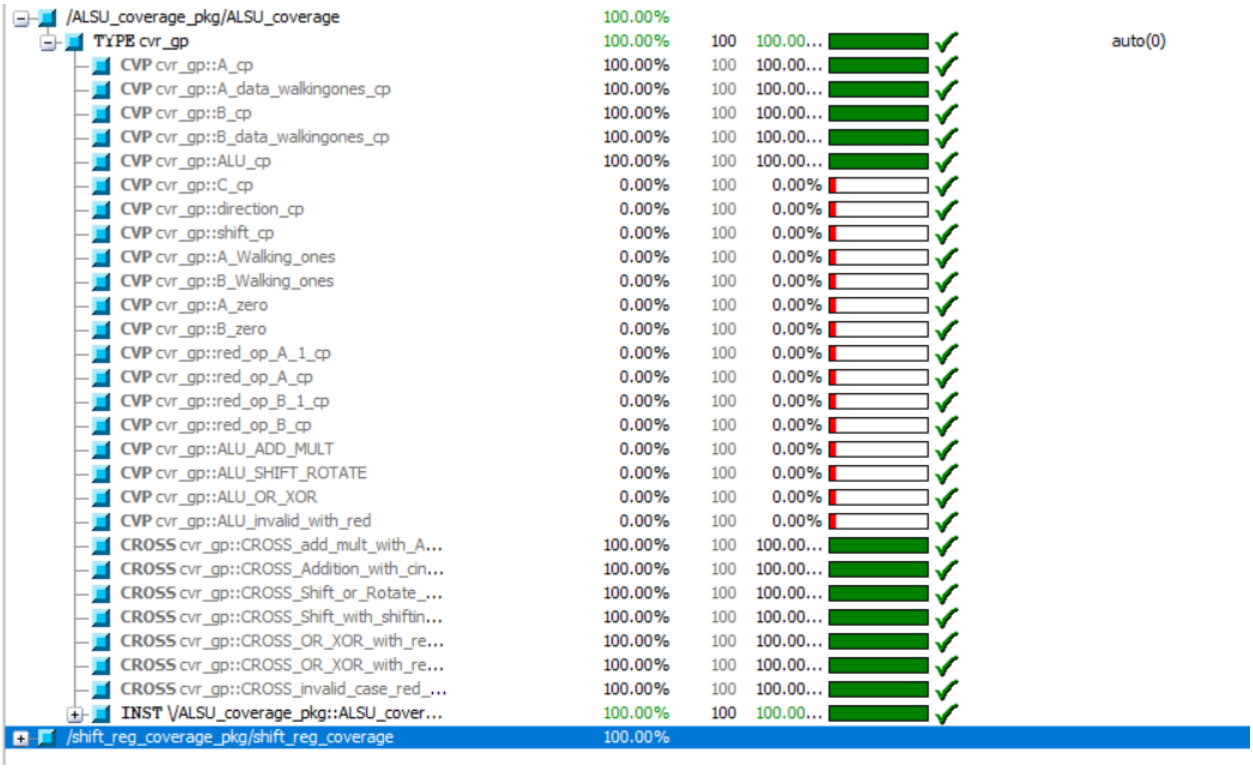
```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         36        36         0   100.00%
```

```
Expression Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Expressions                      8         8         0   100.00%
```

Function Coverage :

```
================================================================================
=== Instance: /ALSU_coverage_pkg
=== Design Unit: work.ALSU_coverage_pkg
================================================================================

Covergroup Coverage:
    Covergroups                     1        na        na   100.00%
        Coverpoints/Crosses        27        na        na        na
            Covergroup Bins       111       111         0   100.00%
```



Assertions:

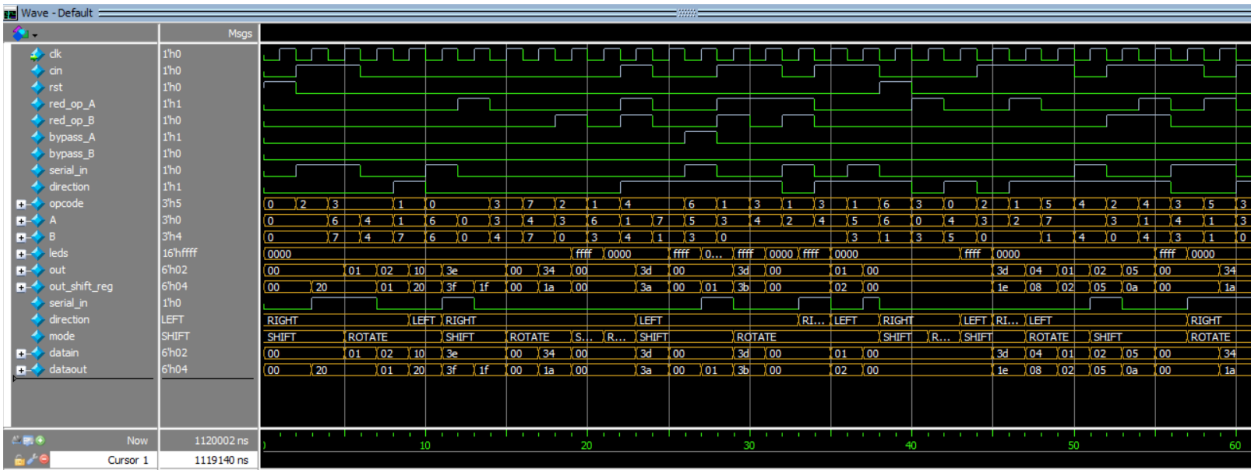
Cover Directives

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/top/alsu/my_assertions/Bypass_A_cover	SVA	✓	Off	9802	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/Bypass_B_cover	SVA	✓	Off	9344	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/invalid_cover	SVA	✓	Off	38053	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_OR_red_A_cover	SVA	✓	Off	17837	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_OR_red_B_cover	SVA	✓	Off	7538	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_OR_cover	SVA	✓	Off	47467	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_XOR_red_A_cover	SVA	✓	Off	17812	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_XOR_red_B_cover	SVA	✓	Off	7617	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_XOR_cover	SVA	✓	Off	47447	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_ADD_cover	SVA	✓	Off	40501	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_MULT_cover	SVA	✓	Off	40203	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_SHIFT_left_cover	SVA	✓	Off	23040	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_SHIFT_right_cover	SVA	✓	Off	23219	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_ROTATE_left_cover	SVA	✓	Off	23343	1	Unli...	1	100%		✓	0	0	0 ns	0
/top/alsu/my_assertions/opcode_ROTATE_right_cover	SVA	✓	Off	23401	1	Unli...	1	100%		✓	0	0	0 ns	0

```
Directive Coverage:
    Directives                  15        15         0   100.00%

DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                 Design Design   Lang File(Line)      Hits Status
                                     Unit   UnitType
--------------------------------------------------------------------------------------------
/top/alsu/my_assertions/Bypass_A_cover   ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(146)
                                                                          9802 Covered
/top/alsu/my_assertions/Bypass_B_cover   ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(147)
                                                                          9344 Covered
/top/alsu/my_assertions/invalid_cover    ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(148)
                                                                          138053 Covered
/top/alsu/my_assertions/opcode_OR_red_A_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(149)
                                                                          17837 Covered
/top/alsu/my_assertions/opcode_OR_red_B_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(150)
                                                                          7538 Covered
/top/alsu/my_assertions/opcode_OR_cover  ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(151)
                                                                          47467 Covered
/top/alsu/my_assertions/opcode_XOR_red_A_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(152)
                                                                          17812 Covered
/top/alsu/my_assertions/opcode_XOR_red_B_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(153)
                                                                          7617 Covered
/top/alsu/my_assertions/opcode_XOR_cover ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(154)
                                                                          47447 Covered
/top/alsu/my_assertions/opcode_ADD_cover ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(155)
                                                                          40501 Covered
/top/alsu/my_assertions/opcode_MULT_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(156)
                                                                          40203 Covered
/top/alsu/my_assertions/opcode_SHIFT_left_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(157)
                                                                          23040 Covered
/top/alsu/my_assertions/opcode_SHIFT_right_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(158)
                                                                          23219 Covered
/top/alsu/my_assertions/opcode_ROTATE_left_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(159)
                                                                          23343 Covered
/top/alsu/my_assertions/opcode_ROTATE_right_cover
                                         ALSU_Assertions Verilog  SVA  ALSU_assertions.sv(160)
                                                                          23401 Covered
```

Waveform :



Transcript :

```
# ** Report counts by severity
# UVM_INFO :   15
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [MONITOR]     1
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [run_phase]    10
# ** Note: $finish    : D:/programs installation files/Questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 1001202 ns  Iteration: 61  Instance: /top
```